

UNIT – 1

1 Mark Questions (Level 1)

1. What is software testing?

Ans. Software testing is the process of verifying software functionality to ensure it meets specified requirements and expected results.

2. Define error in software testing.

Ans. An error is a human mistake made during requirement understanding, design, or coding that leads to incorrect program behaviour.

3. What is a fault?

Ans. A fault is an incorrect logic or implementation in software caused by errors, resulting in unintended application behaviour.

4. What is a defect?

Ans. A defect is a deviation between expected and actual results when software does not meet specified requirements.

5. What is a failure in software?

Ans. A failure occurs when software fails to perform its intended function during execution due to existing defects.

6. What is the main objective of testing?

Ans. The main objective of testing is to detect defects early and ensure quality, reliability, and correct software functionality.

7. What does the V-Model represent?

Ans. The V-Model represents a development approach where testing activities run parallel with corresponding development phases.

8. What is test execution?

Ans. Test execution is the process of running test cases to compare actual outcomes with expected results.

9. What is test automation?

Ans. Test automation uses software tools to execute test cases automatically, reducing manual effort and increasing testing efficiency.

10. What are test tools?

Ans. Test tools are software applications that assist in planning, executing, monitoring, and managing software testing activities.

1 Mark Questions (Level 2)

1. Why is software testing needed?

Ans. Software testing is needed to identify defects early, ensure software quality, reduce failure risks, and verify that the application meets user and business requirements.

2. What is the difference between an error and a defect?

Ans. An error is a human mistake during requirement understanding or coding, whereas a defect is the incorrect software behavior produced due to that error.

3. How is a fault different from a failure?

Ans. A fault is an internal code or logic issue, while a failure occurs when the software does not function correctly during actual execution.

4. Why can testing not prove software is completely bug-free?

Ans. Testing cannot prove software is completely bug-free because exhaustive testing is impossible due to infinite input combinations, execution paths, and practical resource limitations.

5. Why is monitoring test execution important?

Ans. Monitoring test execution is important to track testing progress, identify risks early, measure quality metrics, and ensure testing activities follow the planned schedule.

6. How does automation help in testing?

Ans. Automation helps in testing by executing repetitive test cases quickly, reducing manual effort, increasing test coverage, and supporting continuous integration and delivery processes.

7. Why is the V-Model called a verification and validation model?

Ans. The V-Model is called a verification and validation model because each development phase is verified and matched with a corresponding validation testing phase.

8. How do test tools improve testing efficiency?

Ans. Test tools improve testing efficiency by automating execution, managing test cases, tracking defects, generating reports, and reducing time and human effort.

9. What is meant by measuring test execution?

Ans. Measuring test execution involves collecting and analyzing metrics such as executed test cases, pass-fail status, defect counts, and test coverage.

10. Why are testing activities performed in stages?

Ans. Testing activities are performed in stages to detect defects early, control risks, manage complexity, and ensure quality throughout the software development lifecycle.

2 Mark Questions (Level 2)

1. Explain the need for software testing.

Ans. Software testing is needed to identify defects early in the development process and prevent costly failures after deployment. It ensures the software meets functional and non-functional requirements, improves quality and reliability, and enhances customer satisfaction. Early testing also reduces maintenance cost and project risk by detecting issues before release.

2. Describe the relationship between error, fault, defect, and failure.

Ans. An error is a human mistake during requirement understanding or coding. This error introduces a fault in the software logic. The fault results in a defect when the software produces incorrect output. When the defective code is executed and causes system malfunction, it leads to failure, usually observed by end users.

3. Explain the objectives of software testing.

Ans. The objectives of software testing include detecting defects early, preventing bugs in future development, and ensuring software quality. Testing aims to improve reliability, reduce risks, and increase customer satisfaction. It also helps minimize maintenance costs and supports continuous improvement of the testing process through feedback and defect analysis.

4. Describe the central issue in testing.

Ans. The central issue in software testing is that exhaustive testing is impossible. Software systems have infinite input combinations, execution paths, and states. Due to limited time, cost, and resources, testers must select and prioritize test cases carefully. Therefore, testing focuses on risk-based coverage rather than complete testing.

5. Explain why exhaustive testing is impossible.

Ans. Exhaustive testing is impossible because software systems can have an extremely large number of inputs, conditions, and execution paths. Testing all possible combinations would require unlimited time and resources. Practical constraints such as deadlines, cost, and manpower force testers to focus only on critical and high-risk test scenarios.

6. Describe the role of monitoring during test execution.

Ans. Monitoring during test execution helps track testing progress against the test plan and schedule. It involves collecting metrics such as test case status, defect counts, and execution coverage. Monitoring enables early identification of risks and deviations,

supports corrective actions, and ensures transparency by reporting testing status to stakeholders.

2 Mark Questions (Level 3)

1. Classify the following as error, fault, defect, or failure: *“A programmer writes a wrong formula in code.”*

Ans. This situation is classified as an **error** because the programmer has made a human mistake while writing the logic. The incorrect formula may introduce a fault in the code, which can later cause a defect or failure when the program is executed and produces incorrect results.

2. A login page crashes when the wrong password is entered. Identify the failure and possible fault.

Ans. The crash of the login page is a **failure** because the system stops functioning during execution. The possible **fault** is improper error handling or incorrect conditional logic in the login module, which fails to handle invalid password input gracefully and causes application termination.

3. Suggest a situation where test automation would be more useful than manual testing.

Ans. Test automation is more useful in regression testing where the same test cases are executed repeatedly after every build. For example, in a web application with frequent updates, automated tests can quickly verify existing functionality, saving time, reducing manual effort, and ensuring consistent test coverage across multiple releases.

4. Apply the V-Model to show which testing phase corresponds to system design.

Ans. In the V-Model, the **system design phase** on the verification side corresponds to **system testing** on the validation side. System testing is designed based on system design specifications and verifies whether the complete integrated system meets both functional and non-functional requirements defined during the system design phase.

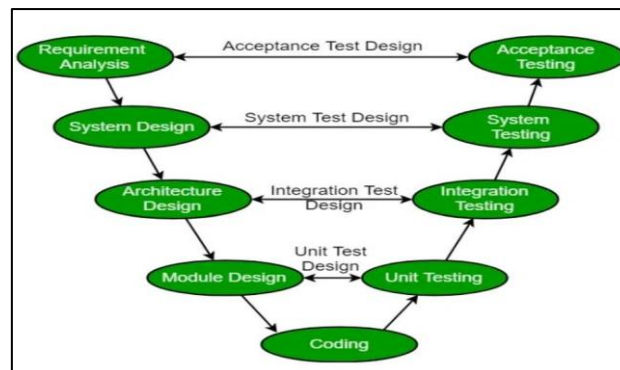
5. Given a project with frequent builds, which testing approach (manual/automated) would you apply and why?

Ans. In a project with frequent builds, **automated testing** is more suitable. Automation allows repeated execution of test cases quickly after each build, supports continuous integration, reduces human effort, and ensures faster feedback. Manual testing would be time-consuming and error-prone under frequent build conditions.

4 Mark Questions (level 3)

1. Apply the V-Model to explain how development phases are linked with testing phases.

Ans. The **V-Model** is a software development model that shows a **direct mapping between development (verification) phases and testing (validation) phases**.



- During **Requirement Analysis**, **Acceptance Test Design** is prepared, which is later used during **Acceptance Testing** to validate user requirements.
- The **System Design** phase corresponds to **System Testing**, where the complete system is tested against functional and non-functional requirements.
- **Architecture Design (High-Level Design)** is linked with **Integration Testing**, ensuring correct interaction and data flow between integrated modules.
- **Module Design (Low-Level Design)** corresponds to **Unit Testing**, where individual modules are tested for correctness.
- **Coding** is performed at the bottom of the V, after which testing activities proceed upward.

This structured linkage ensures **early test planning, defect prevention, and continuous verification and validation** throughout the software development lifecycle.

2. Illustrate with an example how an error leads to a failure in software.

Ans. An error occurs when a developer makes a mistake while understanding requirements or writing program logic. For example, a developer misunderstands the requirement for validating a login password and writes incorrect conditional logic. This human mistake introduces a fault in the source code. When the application is executed, the faulty logic allows invalid inputs to be processed incorrectly, which appears as a defect during testing. If the defect is not detected and the software is released, the login module may crash or behave unexpectedly during real usage. This malfunction experienced by the user is known as a software failure. Thus, a simple human error can propagate through fault and defect stages and finally result in system failure.

3. Show how monitoring and measurement can improve test execution quality.

Ans. Monitoring and measurement play a crucial role in improving the quality of test execution by continuously tracking testing activities and outcomes.

Monitoring ensures that test execution progresses according to the planned schedule by observing test case execution status, defect trends, and resource usage.

Measurement involves collecting quantitative metrics such as pass–fail ratios, defect density, test coverage, and execution effort. By analyzing these metrics, testers can identify delays, quality gaps, and high-risk modules at an early stage.

For example, if monitoring shows a sudden increase in defects in a payment module, additional test cases can be designed and executed for that area. Thus, monitoring and measurement enable timely corrective actions, better control over testing activities, and improved overall test execution quality.

4. Demonstrate how test automation can be integrated into a testing lifecycle.

Ans. Test automation can be smoothly integrated into the testing lifecycle by aligning automated activities with each testing phase.

During the test design phase, suitable test cases such as repetitive, regression, and data-driven tests are identified for automation. In the test development phase, automated test scripts are created using appropriate tools based on the application requirements.

During test execution, these automated scripts are executed repeatedly, especially after new builds, to validate existing functionality efficiently. The automation tools generate execution logs and reports, which are analyzed during test monitoring and control to evaluate quality and progress.

For example, in a project with frequent software builds, automated regression tests can be triggered after every build to quickly detect defects. This integration reduces manual effort, improves test coverage, increases speed, and ensures consistent testing across the lifecycle.

4 Mark Questions (level 4)

1. Differentiate between error, fault, defect, and failure with suitable examples.

Ans.

Basis	Error	Fault	Defect	Failure
Meaning	A human mistake made during requirement understanding or coding	An incorrect logic or statement present in the code	A deviation between expected and actual output	Inability of software to perform its intended function
Who causes it	Developer or designer	Introduced due to developer's error	Observed by tester during testing	Experienced by end user
Stage detected	Requirement/design/coding phase	Coding phase	Testing phase	Execution or production phase
Nature	Human action	Code-level issue	Functional issue	Runtime malfunction
Visibility	Not directly visible	Internally present in code	Visible during testing	Visible to users
Example	Developer misunderstands password validation rule	Wrong conditional logic written	Login accepts invalid password	Application crashes during login

2. Analyse the advantages and limitations of test automation.

Ans. Advantages of Test Automation:

1. **Faster Execution** – Automated tests execute much faster than manual tests, especially for regression testing.
2. **Increased Test Coverage** – Automation allows repeated execution of large test suites, improving coverage.
3. **Reduced Human Error** – Automated tests follow scripts exactly, avoiding manual mistakes.

4. **Supports CI/CD** – Automation enables frequent testing in continuous integration and delivery environments.

Limitations of Test Automation:

1. **High Initial Cost** – Automation tools, script development, and maintenance require high initial investment.
2. **Not Suitable for All Tests** – Usability and exploratory testing cannot be fully automated.
3. **Maintenance Effort** – Test scripts need frequent updates when application changes.
4. **Requires Skilled Resources** – Automation needs testers with programming and tool knowledge.

3. Compare manual testing and automated testing in terms of cost, effort, and reliability.

Ans.

Aspect	Manual Testing	Automated Testing
Cost	Low initial cost but higher long-term cost due to repeated human effort	High initial cost for tools and script development, lower cost in the long run
Effort	Requires continuous human involvement and is time-consuming	Requires less human effort after automation setup
Execution Speed	Slow execution, especially for repetitive test cases	Very fast execution of repetitive and regression tests
Reliability	Less reliable due to human errors and fatigue	More reliable as tests run consistently without human intervention
Best suited for	Exploratory, usability, and ad-hoc testing	Regression, load, and repetitive testing

Example:

In a project with frequent software builds, manual testing becomes time-consuming and error-prone, whereas automated regression tests can be executed after every build, ensuring faster and more reliable testing.

4. Analyse the limitations of software testing and explain why testing alone cannot ensure quality.

Ans. Software testing plays an important role in identifying defects and improving software reliability, but it has several inherent limitations. Testing focuses on evaluating the software after it is developed, but due to practical constraints, testing alone cannot ensure overall software quality.

Limitations of Software Testing:

- **Exhaustive testing is impossible** because software has infinite input combinations and execution paths.
- **Time and resource constraints** limit the extent and depth of testing activities.
- **Testing depends on test data**, which may not cover all real-world usage scenarios.

Conclusion: For example, an application may pass all test cases but still fail in production due to unexpected user behaviour. Therefore, testing alone cannot ensure quality.

5. Examine the V-Model and analyze its strengths and weaknesses in real-world projects.

Ans. The V-Model is a software development model that emphasizes a strong relationship between development (verification) phases and corresponding testing (validation) phases. It promotes early test planning and systematic testing by associating each development activity with a matching testing activity.

Strengths of the V-Model:

- Encourages **early defect detection** by planning testing activities alongside development phases.
- Provides **clear phase-to-phase mapping**, making the process easy to understand and manage.
- Improves **test coverage and traceability** between requirements and test cases.

Weaknesses of the V-Model:

- **Rigid structure** makes it difficult to accommodate changing requirements.
- Not suitable for **large, complex, or evolving systems**.
- **Late visibility of working software**, as coding happens only after design phases.

Conclusion: For example, the V-Model works well in a payroll system with fixed requirements but performs poorly in agile web applications where requirements change frequently. Hence, while the V-Model ensures quality through structured verification and validation, its rigidity limits its effectiveness in dynamic real-world projects.

UNIT – 2

1 Mark Questions (Level 1)

1. What is unit testing?

Ans. Unit testing is the process of testing the smallest units of code, such as functions or methods, individually to verify their correctness and expected behaviour.

2. Define static unit testing.

Ans. Static unit testing is a testing method performed without executing the code, mainly using reviews and static analysis to detect defects early.

3. What is dynamic unit testing?

Ans. Dynamic unit testing is performed by executing the code with test cases to analyse its behaviour and verify outputs against expected results.

4. What is mutation testing?

Ans. Mutation testing is a white-box testing technique where small changes are made in code to evaluate the effectiveness of test cases.

5. What is debugging?

Ans. Debugging is the process of identifying, analysing, and fixing defects or bugs found in the software during testing or execution.

6. What is a test case in unit testing?

Ans. A test case in unit testing is a set of inputs, execution conditions, and expected results used to verify a specific unit of code.

7. What is a test driver?

Ans. A test driver is a piece of code used to call and execute a unit under test, supplying inputs and capturing outputs.

8. What is a test stub?

Ans. A test stub is a temporary piece of code that simulates the behaviour of missing or dependent modules during unit testing.

9. Name any one unit testing tool.

Ans. JUnit is a commonly used unit testing tool for testing Java applications.

1 Mark Questions (Level 2)

1. Why is unit testing important in software development?

Ans. Unit testing is important because it verifies individual units early, detects defects quickly, improves code reliability, and reduces the cost of fixing bugs at later stages.

2. How is static unit testing different from dynamic unit testing?

Ans. Static unit testing checks code without execution using reviews or analysis, whereas dynamic unit testing executes code with test cases to validate runtime behaviour.

3. Why is mutation testing used?

Ans. Mutation testing is used to evaluate the effectiveness of test cases by introducing small code changes and checking whether existing tests detect the errors.

4. How does unit testing help in defect prevention?

Ans. Unit testing helps in defect prevention by identifying logic errors early, ensuring correct implementation, and preventing faulty code from propagating to later development stages.

5. Why is debugging needed after testing?

Ans. Debugging is needed after testing to locate the root cause of detected defects and fix the faulty code responsible for the observed failures.

6. How do stubs help in unit testing?

Ans. Stubs help in unit testing by simulating the behaviour of dependent modules, allowing the unit under test to be tested independently.

7. Why are drivers required in unit testing?

Ans. Drivers are required in unit testing to call and execute a unit under test when higher-level modules are not yet developed.

8. How do unit testing tools improve productivity?

Ans. Unit testing tools improve productivity by automating test execution, reducing manual effort, quickly detecting defects, and supporting repeated regression testing.

9. Why is early defect detection important?

Ans. Early defect detection is important because defects are easier and cheaper to fix in early stages, reducing rework and improving overall development efficiency.

10. How does unit testing improve software quality?

Ans. Unit testing improves software quality by ensuring each unit works correctly, increasing reliability, reducing failures, and strengthening the overall software structure.

2 Mark Questions (Level 2)

1. Explain the concept of unit testing.

Ans. Unit testing is the process of testing the smallest parts of a software application, such as individual functions or methods, in isolation. It is usually performed by developers to verify that each unit works correctly according to the intended logic. Unit testing helps detect errors early, improves code quality, and prevents defects from spreading to higher testing levels.

2. Describe static unit testing with examples.

Ans. Static unit testing is a testing technique performed without executing the code. It involves reviewing source code, design documents, and requirements to identify defects early. Common techniques include reviews, inspections, and static code analysis. For example, detecting unused variables or syntax errors through code inspection before compilation is static unit testing.

3. Explain dynamic unit testing.

Ans. Dynamic unit testing is performed by executing the code with selected test cases to observe its runtime behaviour. It verifies whether the actual output matches the expected output for given inputs. Dynamic unit testing helps identify logical errors and runtime issues. For example, executing a function with valid and invalid inputs to verify correct results is dynamic unit testing.

4. Describe the role of defect prevention in unit testing.

Ans. Defect prevention in unit testing focuses on avoiding defects rather than just detecting them. It includes activities such as requirement analysis, code reviews, static testing, and root cause analysis. By identifying mistakes early at the unit level, defect prevention reduces rework, lowers cost, and improves software quality before defects propagate to later development stages.

5. Explain the need for debugging in software development.

Ans. Debugging is needed in software development to identify and fix the root cause of defects detected during testing. While testing only reveals the presence of bugs, debugging helps analyze faulty logic and correct the code. For example, when a unit test fails, debugging tools are used to trace execution and fix the error.

6. Describe mutation testing in brief.

Ans. Mutation testing is a white-box testing technique used to evaluate the effectiveness of test cases. It involves making small changes, called mutations, in the

source code and checking whether existing test cases detect these changes. If test cases fail for mutated code, they are considered effective, indicating good test coverage.

2 Mark Questions (Level 3)

1. Identify whether the following is static or dynamic unit testing: “Reviewing source code for logical errors.”

Ans. Reviewing source code for logical errors is **static unit testing** because the code is examined without executing it. This activity involves code reviews or inspections to identify defects early. For example, checking conditional statements or loops for incorrect logic during a peer review is a form of static unit testing.

2. Suggest how stubs would be used when testing a function that depends on a database.

Ans. When a function depends on a database, a **stub** is used to simulate the database behaviour during unit testing. The stub returns predefined data instead of accessing the real database. For example, a stub can return sample user records so the function can be tested independently without database connectivity.

3. Apply mutation testing to check the correctness of a conditional statement.

Ans. In mutation testing, small changes are made to a conditional statement to test the effectiveness of test cases. For example, changing a condition from if (a < b) to if (a > b) creates a mutant. If existing test cases detect this change and fail, the condition is correctly tested; otherwise, new test cases are required.

4. Given a module with multiple function calls, show how drivers can help in testing.

Ans. When a module contains multiple function calls and higher-level modules are not ready, **drivers** are used to test individual functions. A driver calls the target function, passes test inputs, and captures outputs. For example, a driver can invoke a calculation function directly to verify its logic without relying on the complete system.

5. Demonstrate how unit testing can prevent defects from reaching later stages.

Ans. Unit testing prevents defects from reaching later stages by validating individual units immediately after development. When a unit fails a test, the defect is fixed before integration. For example, detecting a calculation error in a function during unit testing prevents the same defect from affecting integration or system testing, reducing rework and cost.

6. Apply debugging steps to fix a function that produces incorrect output.

Ans. To fix incorrect output, debugging begins by identifying the failing test case and reproducing the error. The code execution is then traced to locate the faulty logic. After correcting the error, the function is re-tested to confirm the fix. For example, correcting a wrong loop condition and re-running the test ensures correct output.

4 Mark Questions (Level 3)

1. Illustrate the process of dynamic unit testing with a suitable example.

Ans. Dynamic unit testing is a testing technique in which individual units are tested by **executing the code** with selected test cases.

- The process starts by **identifying the unit** to be tested, such as a function or method.
- **Test cases are designed** based on the unit's expected functionality and input conditions.
- The unit is then **executed in isolation**, using **drivers** to call the unit and **stubs** to simulate dependent modules if required.
- The **actual output is compared** with the expected output to identify defects.
- If defects are found, the code is **debugged, corrected, and re-tested** until the unit behaves correctly.

Example: A function that calculates the total price is executed with valid and invalid input values. If the outputs match the expected results, the unit is considered correctly tested.

2. Show how stubs and drivers are used together during unit testing.

Ans. During unit testing, stubs and drivers are used to isolate the unit under test when dependent or higher-level modules are unavailable.

A **driver** is a temporary program that calls the unit under test, supplies test inputs, and captures outputs.

A **stub** simulates the behavior of lower-level modules that are called by the unit under test.

For example, while testing a payment processing function, a driver can invoke the function with test data, while stubs simulate responses from external services such as databases or payment gateways.

This combined use allows the unit to be tested independently, ensuring correctness without relying on the complete system.

3. Demonstrate the steps involved in mutation testing with an example.

Ans. Mutation testing is a white-box testing technique used to evaluate the effectiveness of unit test cases by intentionally introducing small changes into the source code and observing test behaviour.

The steps involved in mutation testing are as follows:

- Small changes, called **mutations**, are introduced into the original program by modifying operators, conditions, or constants.
- Each modified version of the program is treated as a **mutant**.
- Existing unit test cases are executed against each mutant.
- If a test case fails, the mutant is said to be **killed**, indicating effective test cases.
- If a mutant survives, it shows that the test cases are inadequate and need improvement.

Example: Changing a condition from if ($x > y$) to if ($x < y$) creates a mutant. If test cases detect this change and fail, the condition is correctly tested.

4. Apply defect prevention techniques during the unit testing phase.

Ans. Defect prevention during the unit testing phase focuses on avoiding defects before they propagate to later stages. This is achieved through techniques such as proper requirement understanding, code reviews, static unit testing, and adherence to coding standards.

Developers analyse defects found during unit testing to identify their root causes and prevent similar mistakes in the future. Practices like peer reviews, use of checklists, and early validation of logic help reduce errors at the source.

For example, reviewing a function's logic before execution can prevent incorrect calculations from entering integration testing. Thus, defect prevention during unit testing reduces rework, lowers cost, and improves overall software quality.

4 Mark Questions (Level 4)

1. Differentiate between static and dynamic unit testing with advantages and limitations.

Ans.

Basis	Static Unit Testing	Dynamic Unit Testing
Definition	Testing performed without executing the code	Testing performed by executing the code
Main objective	Defect prevention by early detection	Defect detection through runtime behaviour
Execution required	No code execution required	Code execution is mandatory
Techniques used	Reviews, inspections, walkthroughs, static analysis	Test case execution, input–output validation
Advantages	Detects defects early, reduces cost, improves code quality	Finds logical and runtime errors, validates functionality
Limitations	Cannot detect runtime or performance issues	Requires test data and more execution time
Example	Reviewing source code to find unused variables	Executing a function with test inputs to verify output

2. Analyse how mutation testing improves the effectiveness of test cases.

Ans. Mutation testing improves test case effectiveness by introducing small changes in the source code, such as modifying operators, conditions, or constants, to create mutants.

- Existing test cases are executed against these mutants to evaluate whether they can detect the introduced faults. Mutants that are detected are considered “killed.”
- Surviving mutants indicate weaknesses in test cases, such as missing boundary conditions, insufficient input coverage, or weak assertions.
- By analysing surviving mutants, testers improve and strengthen test cases to cover logical paths more effectively, increasing fault detection capability and overall software quality.

Example: Changing a condition from if ($x \geq y$) to if ($x > y$) may expose missing boundary value test cases.

3. Compare unit testing and debugging in terms of purpose and process.

Ans.

Aspect	Unit Testing	Debugging
Purpose	To detect defects in individual units of software	To identify and fix the root cause of detected defects
Focus	Verifying correctness of code functionality	Analyzing faulty logic and correcting errors
Performed by	Primarily developers	Developers
When performed	Immediately after unit development	After a test failure is observed
Process	Designing test cases, executing code, comparing expected and actual outputs	Tracing execution, isolating faulty code, correcting defects
Outcome	Identification of defects	Removal of defects

Example: During unit testing, a function produces incorrect output for a given input, indicating a defect. Debugging is then used to trace the code, locate the faulty logic, and fix the issue so the unit passes the test.

4. Analyse the role of unit testing tools in improving test coverage and efficiency.

Ans.

- Unit testing tools improve test coverage by allowing automated execution of a large number of test cases across different inputs, conditions, and code paths.
- These tools provide reports on test execution and coverage, helping developers identify untested areas of code and improve test completeness.
- Unit testing tools increase efficiency by reducing manual effort, saving time, and enabling repeated execution of tests with minimal human intervention.
- Faster feedback from automated tests helps developers detect and fix defects early, improving overall software quality.

Example: Tools like **JUnit** allow automatic execution of unit tests after every code change, ensuring quick defect detection and consistent testing.

5. Examine the impact of effective unit testing on overall software quality.

Ans.

- Effective unit testing improves software quality by detecting and fixing defects early at the unit level, preventing them from propagating to integration and system testing stages.
- It enhances code reliability and stability by ensuring that each individual unit functions correctly before being combined with other modules.
- Unit testing promotes better design and maintainability by encouraging modular, well-structured, and testable code, reducing future debugging and rework.
- Early validation through unit testing increases developer confidence and reduces failure rates in later testing phases.

Example:

If a calculation function is thoroughly unit tested, errors are eliminated early, preventing incorrect results in dependent modules and improving overall system quality.
